



ADVANCED DATABASE SYSTEMS

WEEK 11

Parallelism in Database Systems



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

Recommended Reading:

Chapter 8 in Principles of Distributed Database Systems.

M. Tamer Özsu · Patrick Valduriez. Springer ISBN 978-3-030-26252-5 2020



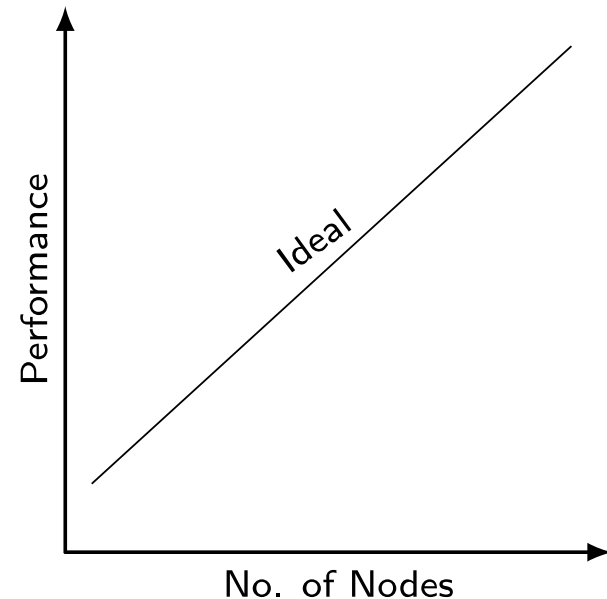
Objectives of Parallel Systems

- High-performance using parallelism
 - High throughput for OLTP loads
 - ▶ Lots of short transactions, which update a few data
 - Low response time for OLAP queries
 - ▶ Large queries, which read lots of data
- High availability and reliability through data replication and recovery
- **Extensibility** and **scalability** by adding resources
 - Processors, memory, disk, network



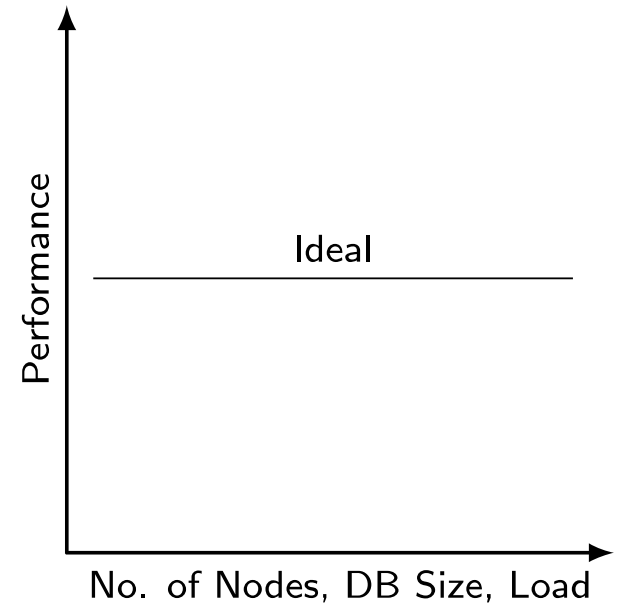
Extensibility

- Ideal: linear speed-up
 - Linear increase of performance by growing the components
 - ▶ For a fixed database size and load



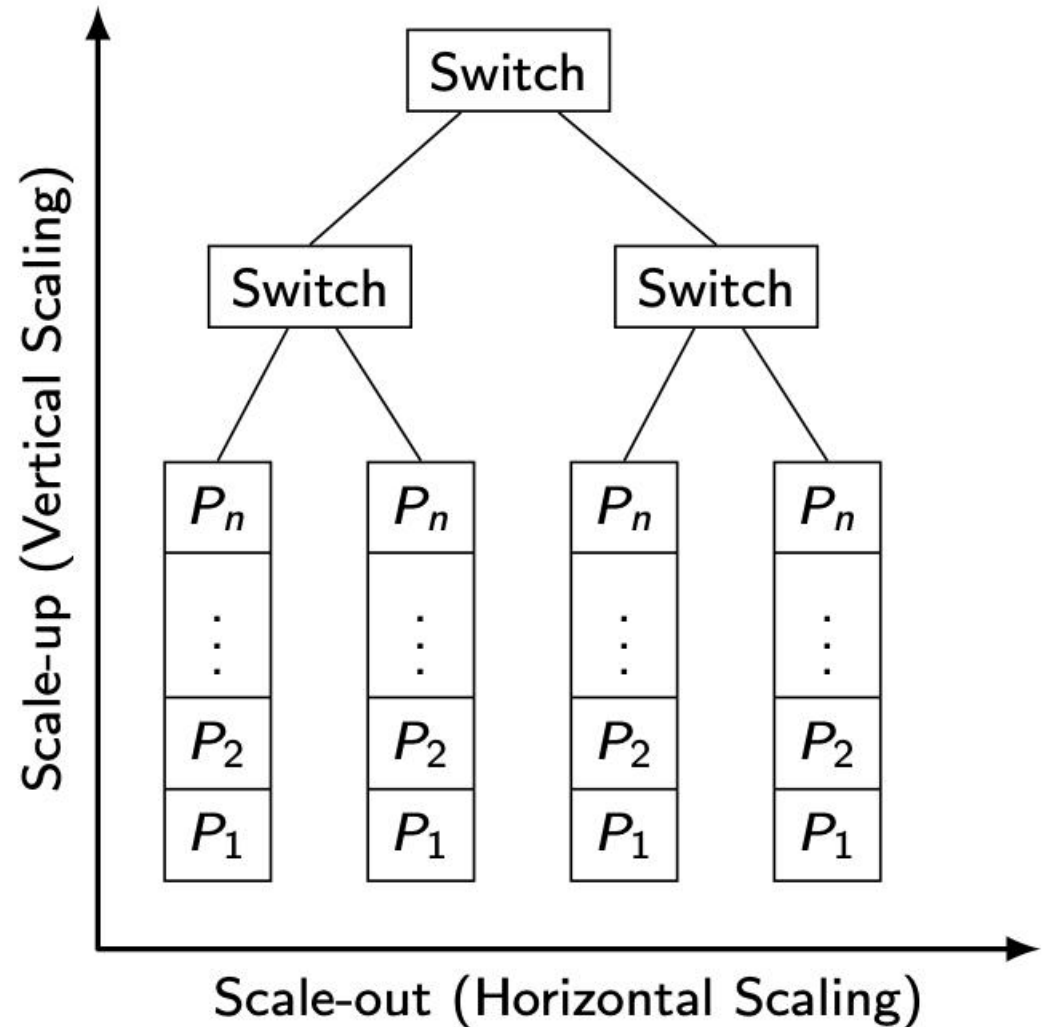
Scalability

- Ideal: linear scale-up
 - Sustained performance for a linear increase of database size and load
 - By proportional increase of components



Vertical vs Horizontal Scale-up

- Typically in a computer cluster



Scaling Traditional Databases

- Traditional RDBMSs can be either scaled:
 - **Vertically** (or **Up**)
 - Can be achieved by hardware upgrades (e.g., faster CPU, more memory, or larger disk)
 - Limited by the amount of CPU, RAM and disk that can be configured on a single machine
 - **Horizontally** (or **Out**)
 - Can be achieved by adding more machines
 - Requires database *sharding* and probably *replication*
 - Limited by the Read-to-Write ratio and communication overhead



Speed-up Limits

- Hardware/software
 - As we add more resources, arbitration conflicts increase
 - ▶ E.g. Access to the bus by processors
- Application
 - Only part of a program can be parallelized
 - Amdahl's law that gives the maximum speed-up



Amdahl's Law

- How much faster will a parallel program run?
 - Suppose that the sequential execution of a program takes T_1 time units and the parallel execution on p processors/machines takes T_p time units
 - Suppose that out of the entire execution of the program, s fraction of it is not parallelizable while $1-s$ fraction is parallelizable
 - Then the speedup (**Amdahl's formula**):

$$\frac{T_1}{T_p} = \frac{T_1}{(T_1 \times s + T_1 \times \frac{1-s}{p})} = \frac{1}{s + \frac{1-s}{p}}$$

Amdahl's Law: An Example

- Suppose that:
 - 80% of your program can be parallelized
 - 4 machines are used to run your parallel version of the program

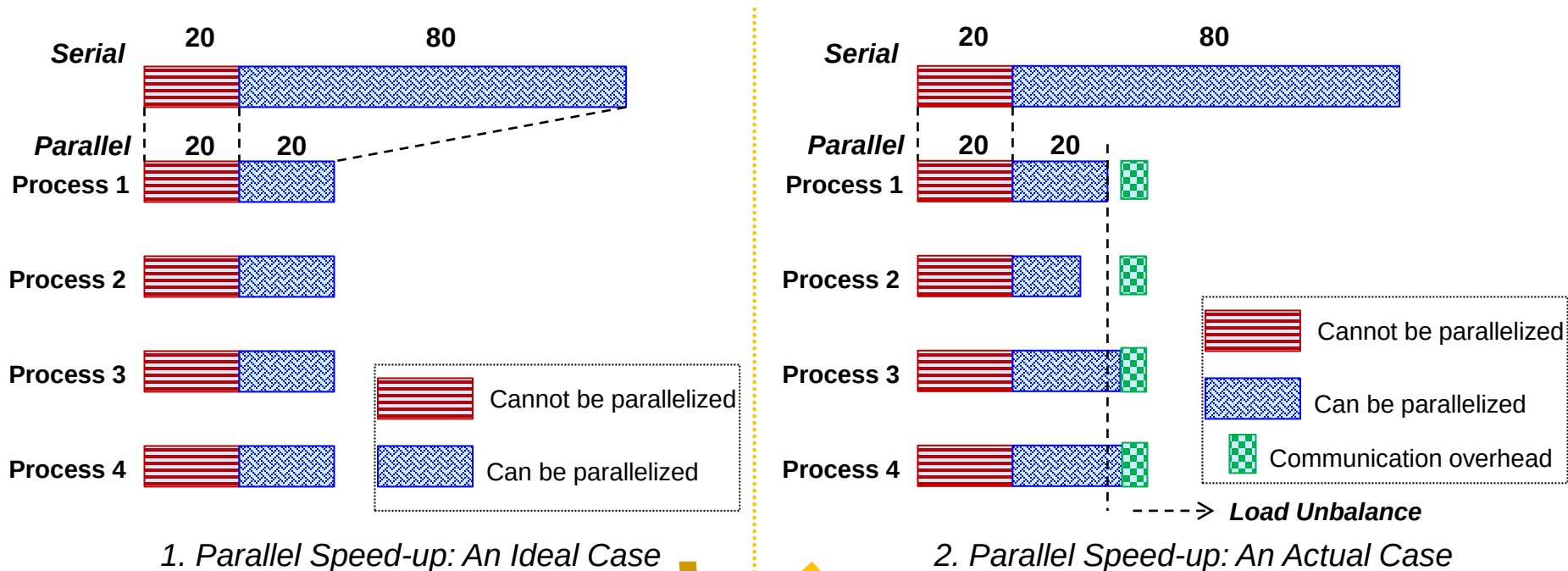
- The speedup you can get according to Amdahl's law is:

$$\frac{1}{s + \frac{1-s}{p}} = \frac{1}{0.2 + \frac{0.8}{4}} = 2.5 \text{ times}$$

Although you use 4 processors you cannot get a speedup more than 2.5 times!

Real Vs. Actual Cases

- Amdahl's argument is too simplified
- In reality, communication overhead and potential workload imbalance exist upon running parallel programs



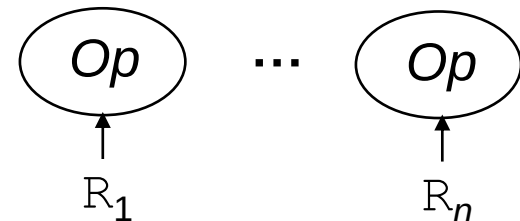
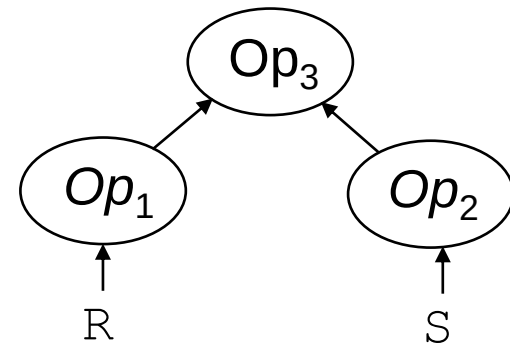
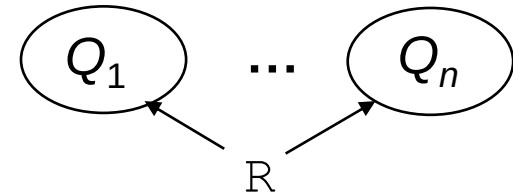
Some Guidelines

- Here are some guidelines to effectively benefit from parallelization:
 1. Maximize the fraction of your program that can be parallelized
 2. Balance the workload of parallel processes
 3. Minimize the time spent for communication



Data-based Parallelism

- Inter-query
 - Different queries on the same data
 - For concurrent queries
- Inter-operation
 - Different operations of the same query on different data
 - For complex queries
- Intra-operation
 - The same operation on different data
 - For large queries



Barriers to Parallelism

- Startup
 - The time needed to start a parallel operation may dominate the actual computation time
- Interference
 - When accessing shared resources, each new process slows down the others (hot spot problem)
- Skew
 - The response time of a set of parallel processes is the time of the slowest one

Parallel data management techniques intend to overcome these barriers

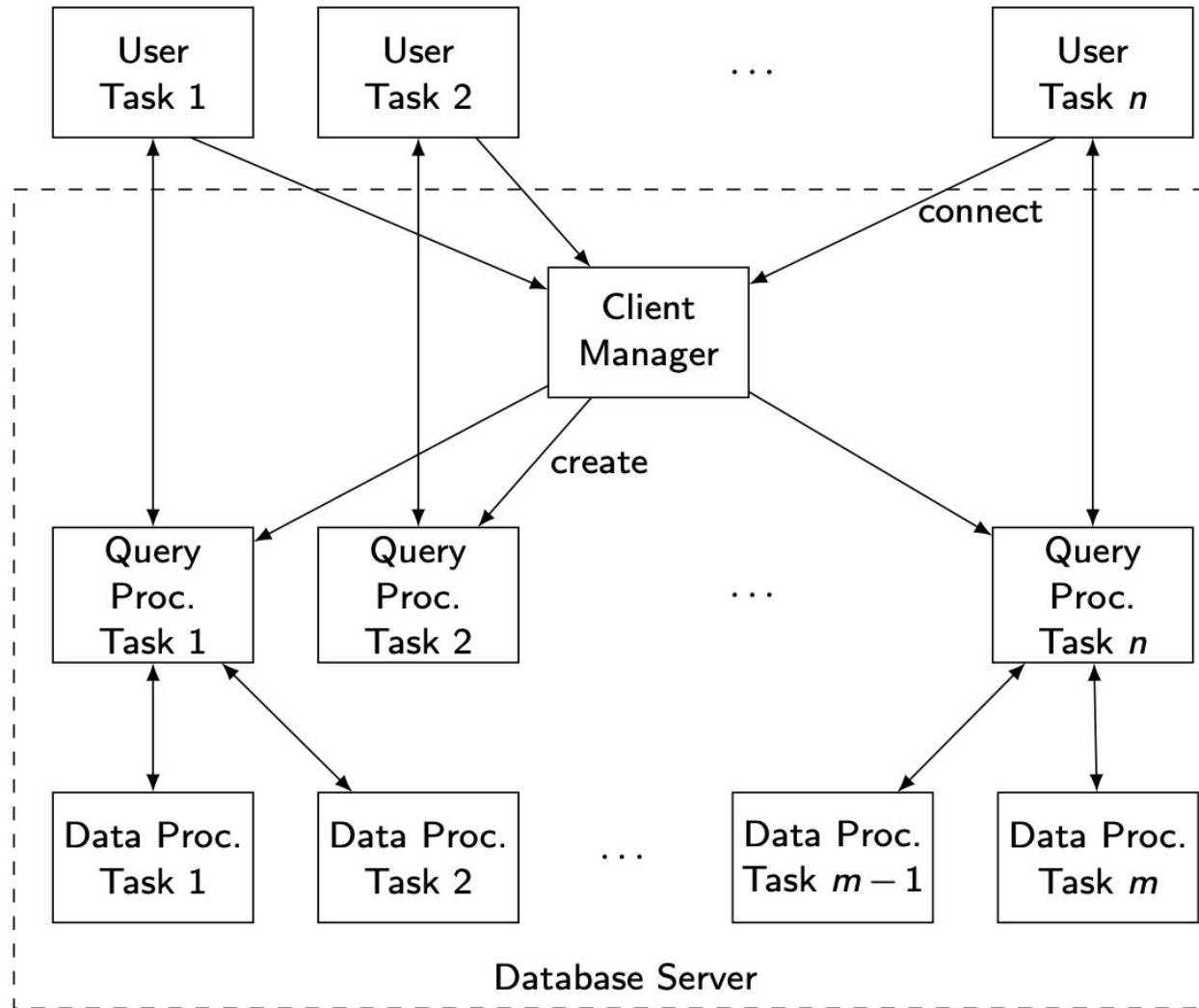


Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters



Parallel DBMS – general architecture



Parallel DBMS Functions

- ❑ Client manager
 - ❑ Support for client interactions and user sessions
 - ❑ Load balancing
- ❑ Query processor
 - ❑ Compilation and optimization
 - ❑ Catalog and metadata management
 - ❑ Semantic data control
 - ❑ Execution control
- ❑ Data processor
 - ❑ Execution of DB operations
 - ❑ Transaction management
 - ❑ Data management



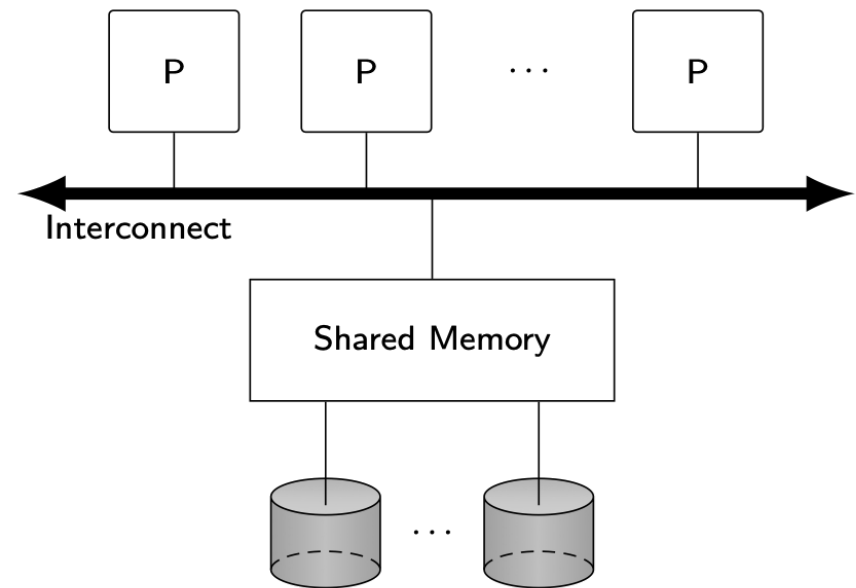
Parallel System Architectures

- Shared memory (SM)
 - Uniform Memory Architecture (UMA)
 - Non-Uniform Memory Architecture (NUMA)
- Shared disk (SD)
- Shared nothing (SN)



UMA

- Physical memory shared by all processors
 - Symmetric multiprocessor (SMP) or multicore processor
 - Constant access time
- Examples
 - XPRS, Volcano, DBS3
- Assessment
 - + Simplicity, load balancing, fast communication
 - vulnerability to memory fault, low extensibility



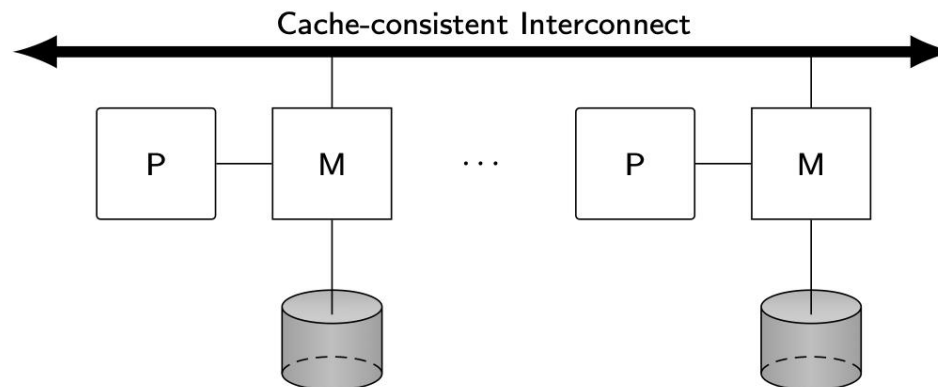
NUMA

- ❑ Shared-memory vs. distributed memory
 - ❑ Mixes two different aspects : addressing and memory
 - ▶ Addressing: single address space vs multiple address spaces
 - ▶ Physical memory: central vs distributed
- ❑ NUMA = single address space on distributed physical memory
 - ❑ Eases application portability
 - ❑ Extensibility
- ❑ The most successful NUMA is Cache Coherent NUMA (CC-NUMA)

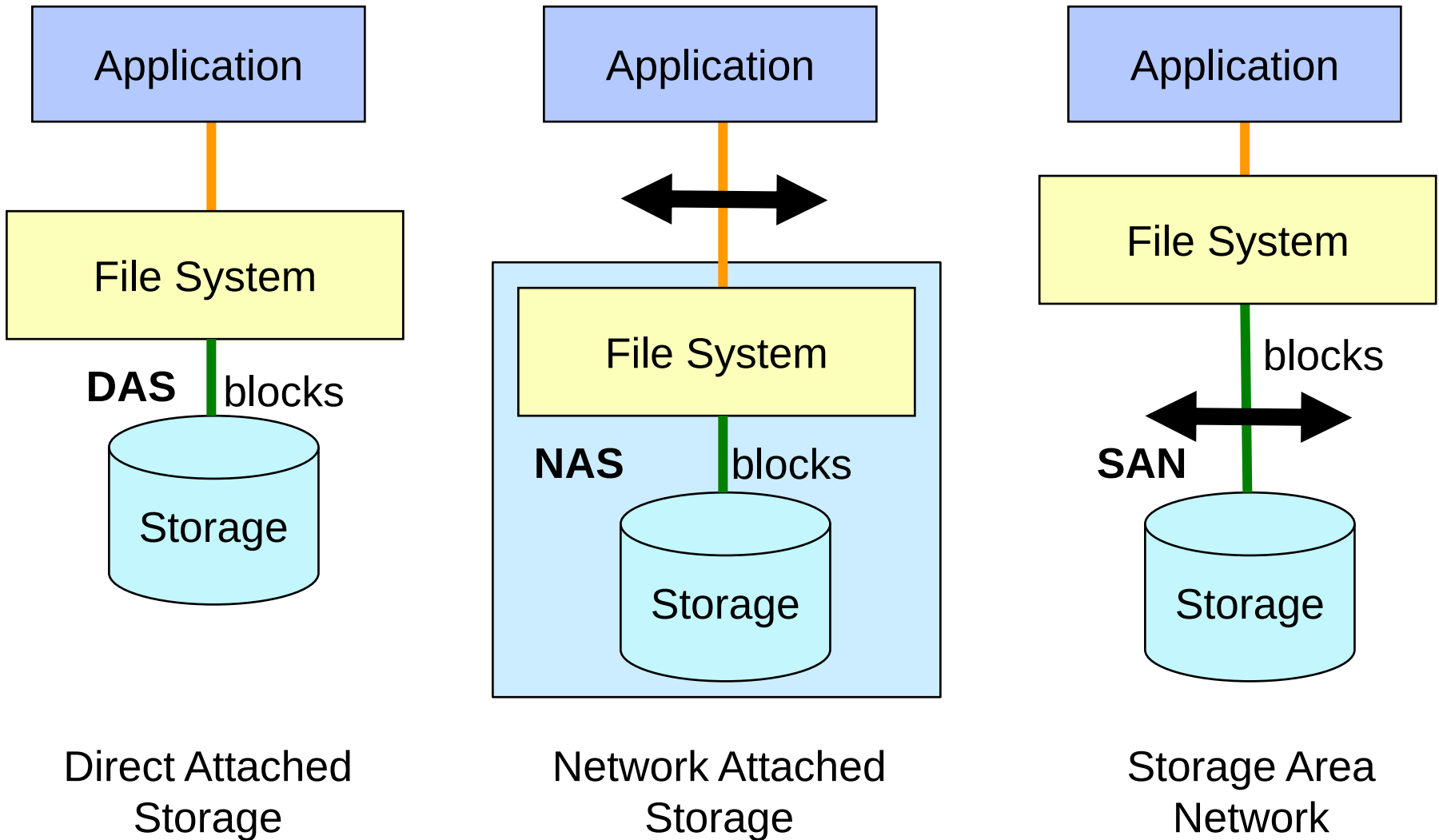


CC-NUMA

- ❑ Principle
 - ❑ Main memory physically distributed (as in shared-nothing)
 - ❑ However, any processor has access to all other processors' memories
- ❑ Cache consistency
 - ❑ Special consistent cache interconnect
 - ▶ Remote memory access very efficient, only a few times (typically between 2 and 3 times) the cost of local access
 - ❑ Exploit Remote Direct Memory Access (RDMA)
 - ▶ Now provided by low latency cluster interconnects such as Infiniband and Myrinet



Storage: DAS vs NAS vs SAN



Shared-Disk

- Shared disk, private memory

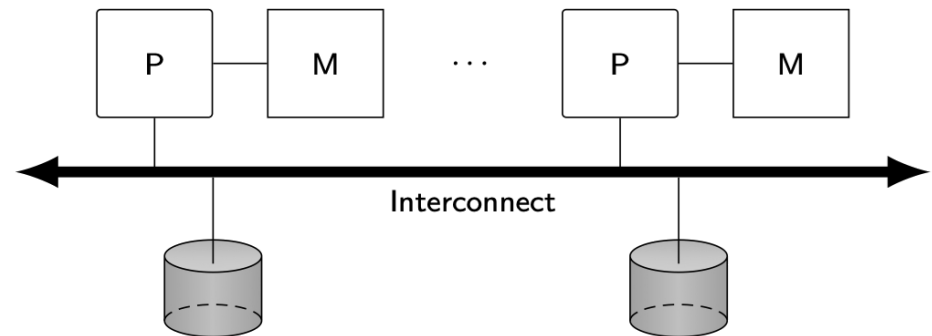
- SAN
- Cache coherency

- Examples

- Oracle RAC Exadata
- IBM PowerHA

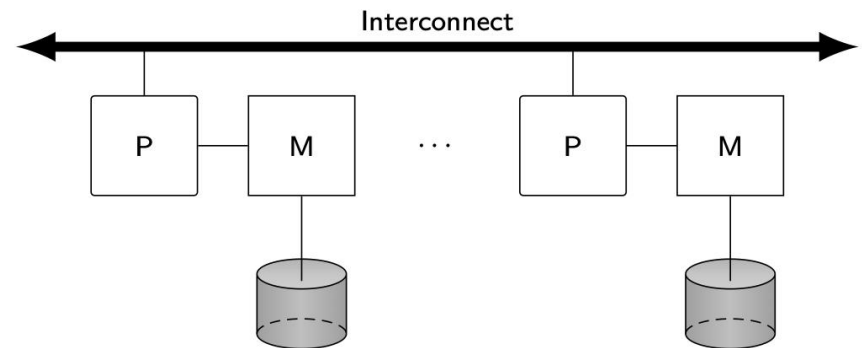
- Assessment

- + Simplicity for admin.
- Network cost (SAN), scalability (caused by the potential bottleneck and overhead of cache coherence protocols)



Shared-Nothing

- No sharing of either disk or memory
 - Data partitioning
- Examples
 - DB2 DPF, SQL Server Parallel DW, Teradata, MySQLcluster
 - NoSQL, NewSQL
- Assessment
 - + Scalability, cost/performance
 - Complex (distributed updates)



Parallel DBMS Techniques

- Data placement
 - Data partitioning
 - Replication
- Parallel query processing
 - Parallel algorithms for relational operators
 - Query optimization
 - Load balancing
- Transaction management
 - Similar to distributed transaction management
 - ▶ But need to scale up to many nodes
 - Fault-tolerance and failover



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters



Data Partitioning

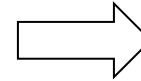
- Each relation is divided in n partitions (subrelations), where n is a function of relation size and access frequency
- Horizontal partitioning
 - Round-robin
 - ▶ Maps i -th element to node $i \bmod n$
 - ▶ Simple but only exact-match queries
 - ▶ Requires full scan
 - Hash
 - ▶ Only exact-match queries but small index
 - Range
 - ▶ Supports range queries but large index



Partitioning Functions

□ Hashing

- (k, v) assigned to node $h(k)$
- Exact match queries
- Problem with skew distribution

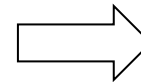


2				
2				
4				
4				
10				
12				

1				
11				
11				
13				

□ Range

- (k, v) to the node that holds k 's interval
- Exact match and range queries
- Needs an index on key



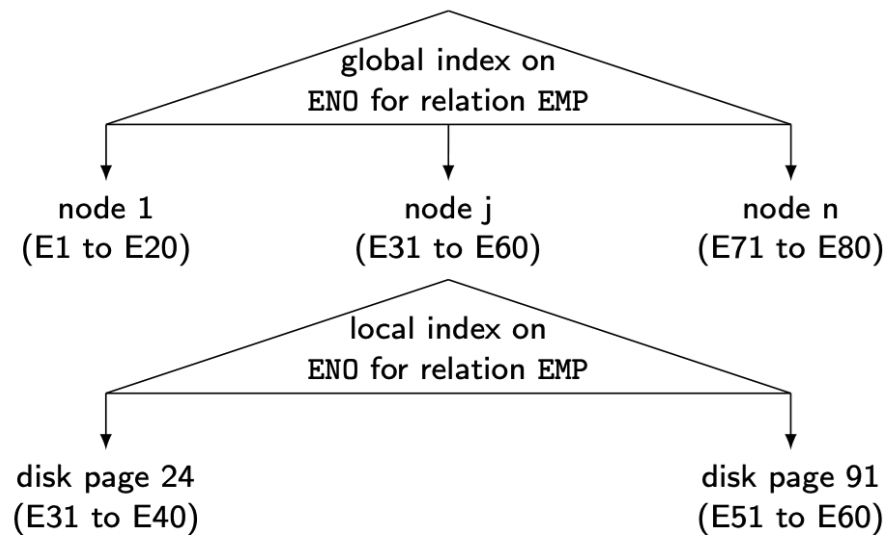
1				
2				
2				
4				
4				

10				
11				
11				
12				
13				



Placement Directory

- Performs mapping function
 - $F(\text{relname, placement attr}) = \text{lognode-id}$
- Global index on placement attr. available at each node
- In addition, each node has its local index (to access disk pages)



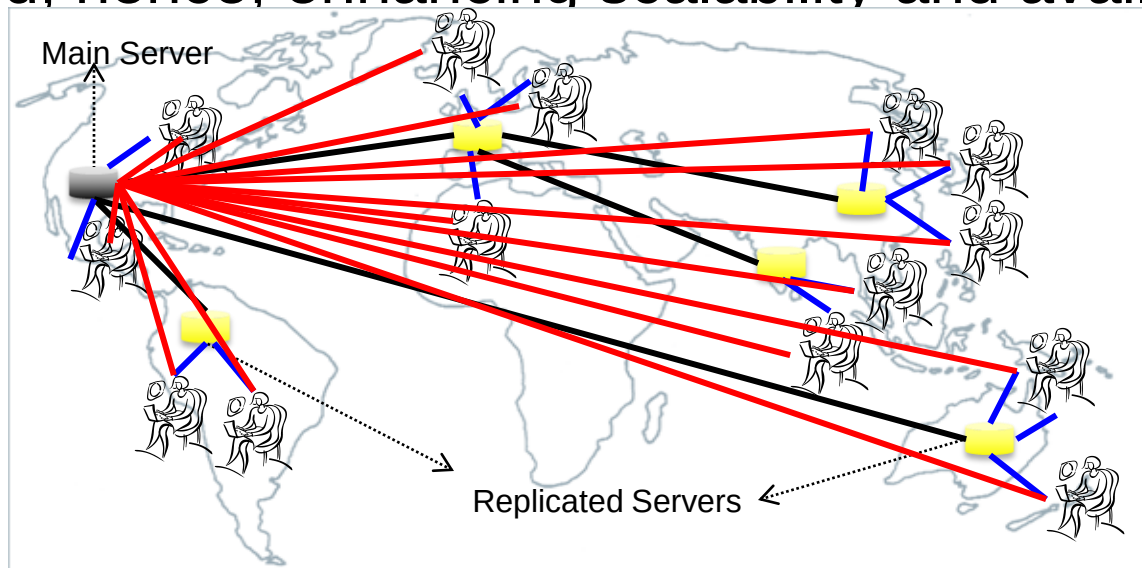
Why Replicating Data?

- Replicating data across servers helps in:
 - Avoiding performance bottlenecks
 - Avoiding single point of failures
 - And, hence, enhancing scalability and availability



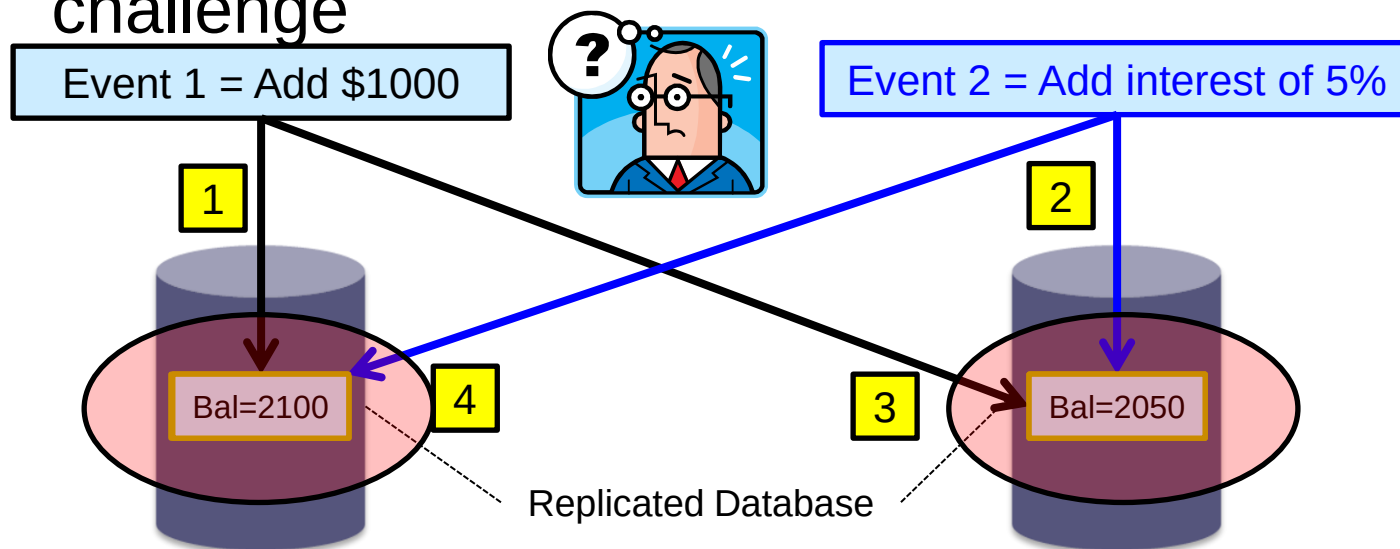
Why Replicating Data?

- Replicating data across servers helps in:
 - Avoiding performance bottlenecks
 - Avoiding single point of failures
 - And, hence, enhancing scalability and availability



But, Consistency Becomes a Challenge

- An example:
 - In an e-commerce application, the bank database has been replicated across two servers
 - Maintaining consistency of replicated data is a challenge



Replicated Data Partitioning

- High-availability requires data replication
 - Simple solution is mirrored disks
 - ▶ Hurts load balancing when one node fails
 - More elaborate solutions achieve load balancing
 - ▶ Interleaved partitioning (Teradata)
 - ▶ Chained partitioning (Gamma)



Interleaved Partitioning

Node	1	2	3	4
Primary copy	R_1	R_2	R_3	R_4
Backup copies	$R_{2,1}$ $R_{3,1}$ $R_{4,1}$	$R_{1,1}$ $R_{3,2}$ $R_{4,2}$	$R_{1,2}$ $R_{2,2}$ $R_{4,3}$	$R_{1,3}$ $R_{2,3}$ $R_{3,3}$



Chained Partitioning

Node	1	2	3	4
Primary copy	R_1	R_2	R_3	R_4
Backup copies	R_3 R_4	R_4 R_1	R_1 R_2	R_2 R_3



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

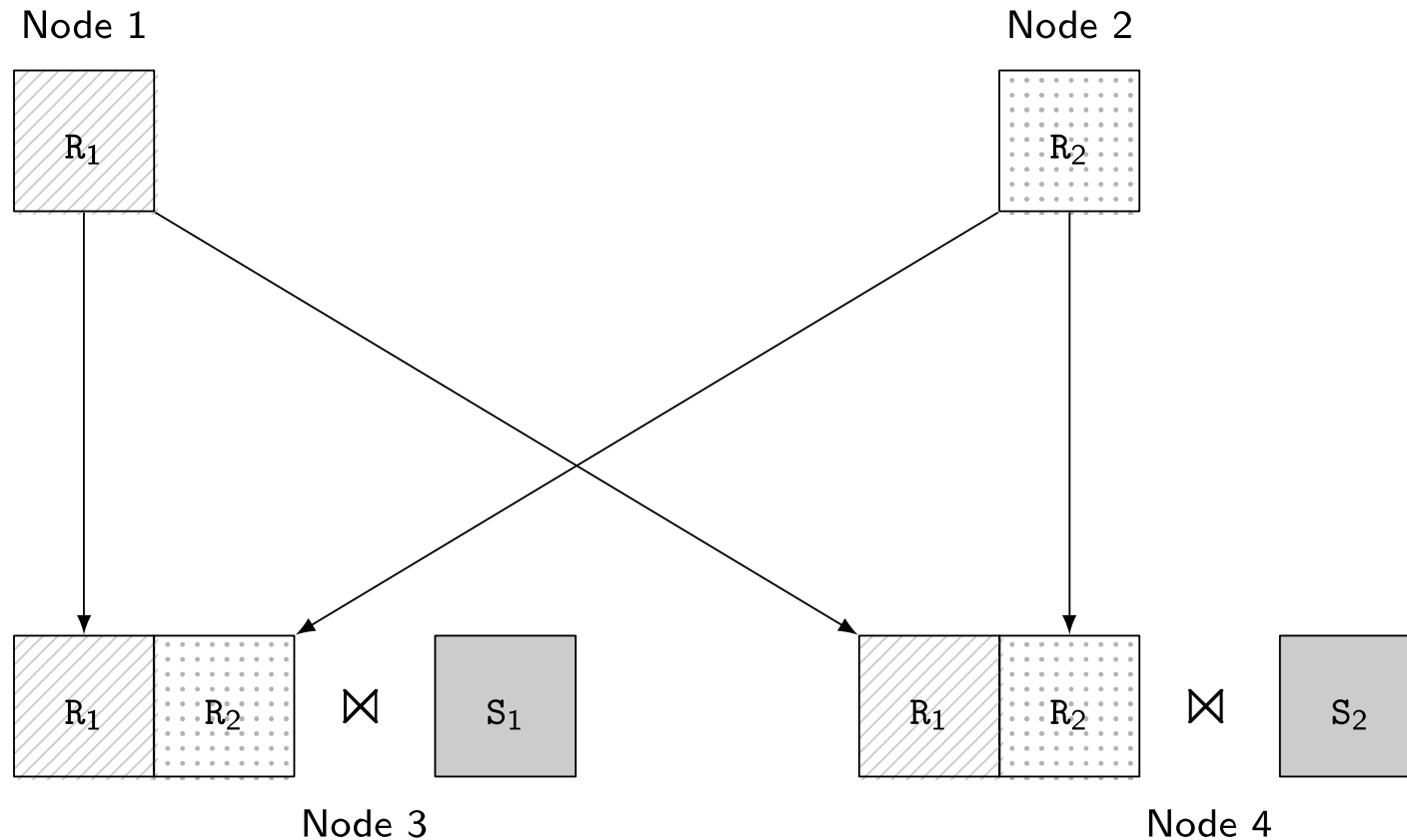


Join Processing

- Two basic algorithms for intra-operator parallelism
 - Parallel nested loop join: no special assumption
 - Parallel hash join: equijoin
- They also apply to other complex operators such as duplicate elimination, union, intersection, etc. with minor adaptation



Parallel Nested Loop Join



$$R \bowtie S = \bigcup_{i=1}^n (R \bowtie S_i)$$

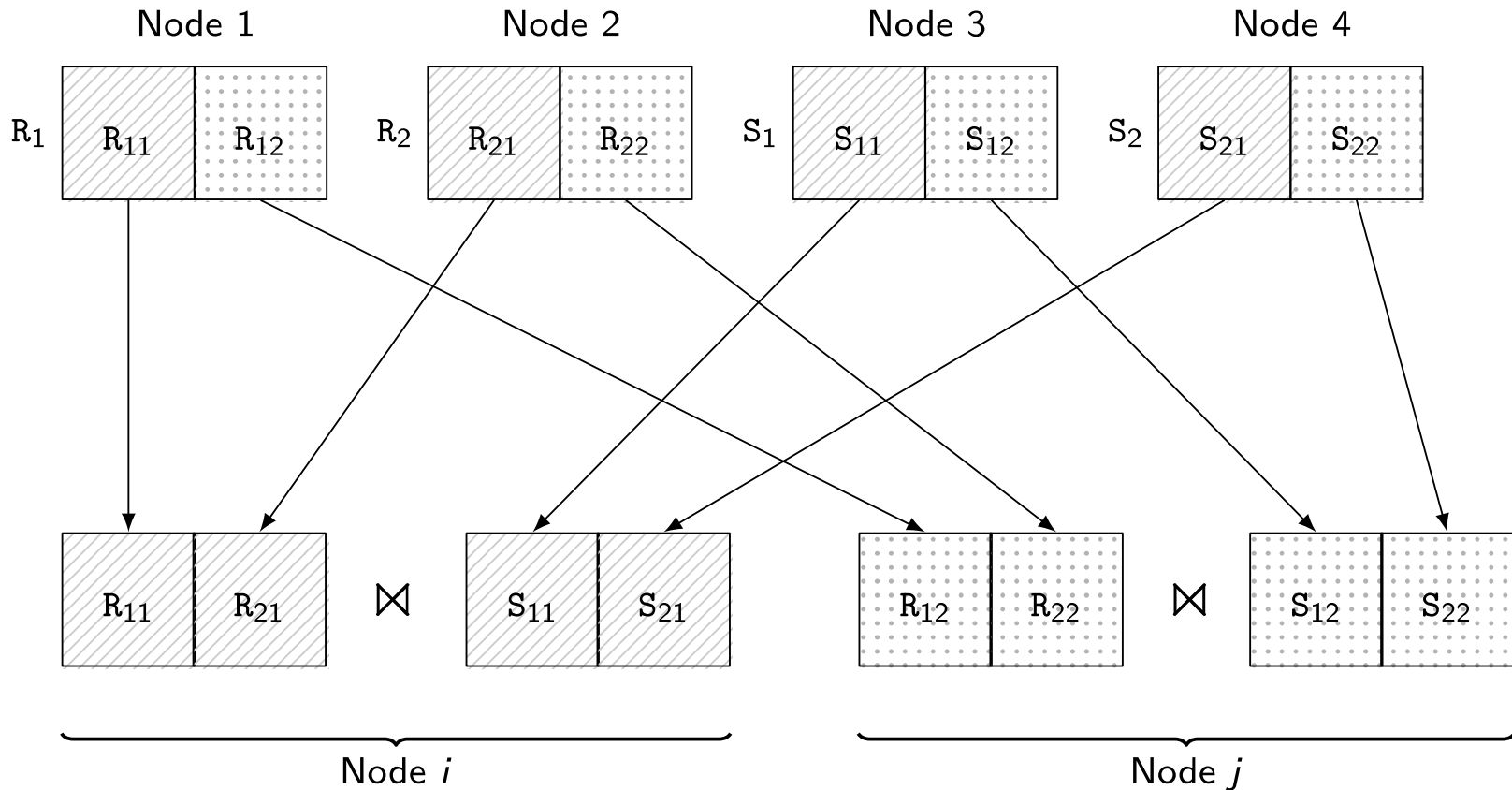


Parallel Nested Loop Join - implementation

- ❑ Two nested loops phase
 - ❑ One relation is chosen as the inner relation (e.g. S), the other relation as the outer relation (e.g. R)
- ❑ First phase
 - ❑ Each fragment of R is sent and replicated at each node that contains a fragment of S (there are n such nodes)
- ❑ Second phase
 - ❑ Each S -node j receives relation R entirely, and locally joins R with its fragment of S
- ❑ Pipelining
 - ❑ As soon as R tuples are received, they can be processed in pipeline by accessing the S tuples, e.g. through an index



Parallel Hash Join



$$R \bowtie S = \bigcup_{i=1}^p (R_i \bowtie S_i)$$

Parallel Hash Join - implementation

- Build phase
 - Hashes R , used as inner relation, on the join attribute
 - Sends it to the target p nodes that build a hash table for the incoming tuples
- Probe phase
 - Sends S , the outer relation, associatively to the target p nodes
 - Each target node probes the hash table for each incoming tuple
- Pipelining
 - As soon as the hash tables have been built for R , the S tuples can be sent and processed in pipeline by probing the hash tables



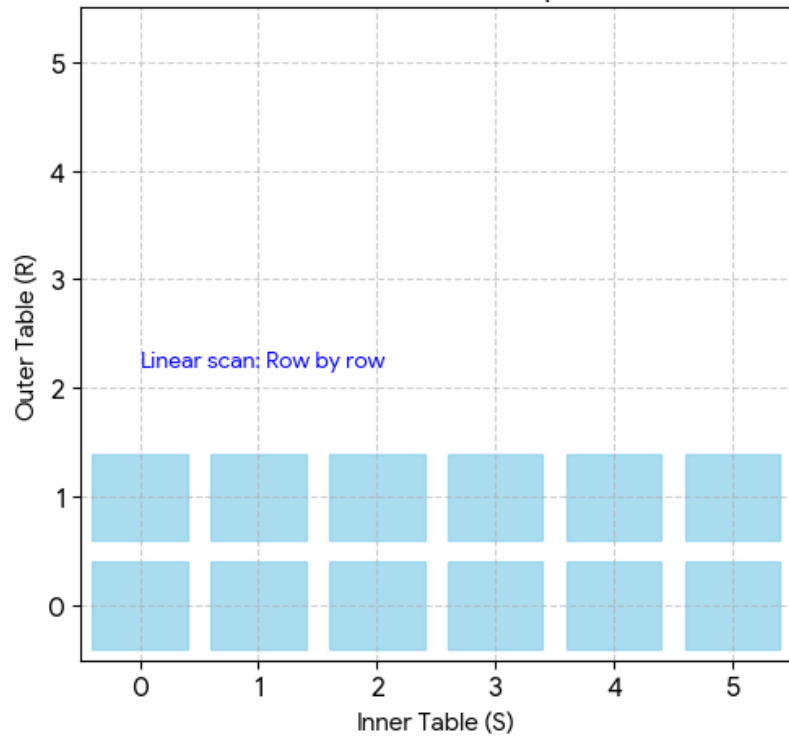
Variants

- ❑ To exploit large main memories and multicore
- ❑ Symmetric hash join
 - ❑ The traditional build and probe phases of the basic hash join algorithm are simply interleaved, using two hash tables
 - ❑ When a tuple arrives
 - ▶ It is used to probe the hash table corresponding to the other relation and find matching tuples
 - ▶ Then, it is inserted in its corresponding hash table so that tuples of the other relation arriving later can be joined
- ❑ Ripple join
 - ❑ Generalization of the nested loop join algorithm where the roles of inner and outer relation continually alternate during query execution

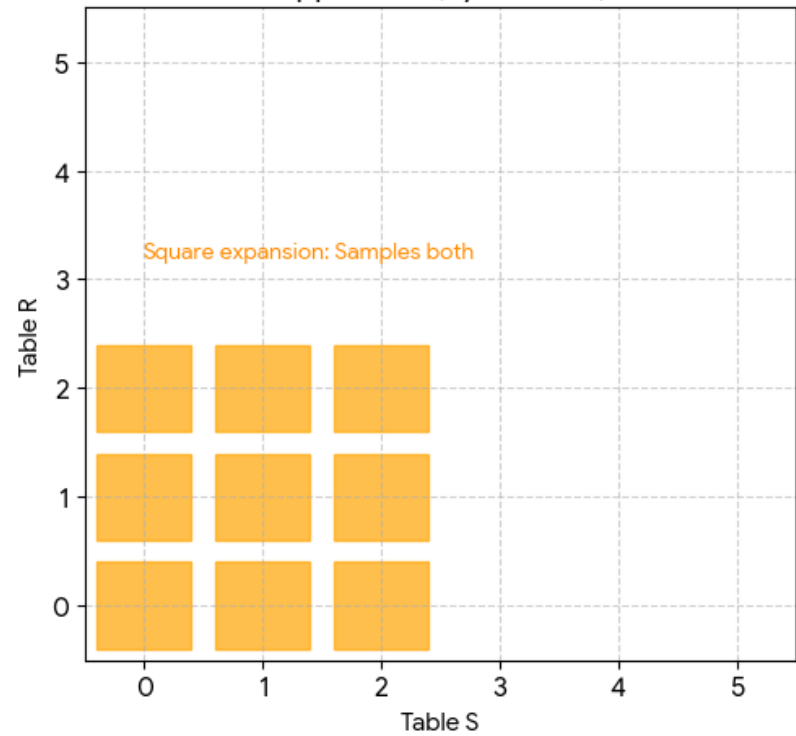


Ripple Join

Standard Nested Loop Join



Ripple Join (Symmetric)



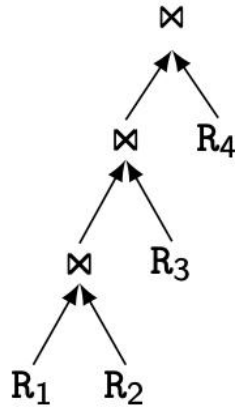
Parallel Query Optimization

- The objective is to select the “best” parallel execution plan for a query using the following components
- Search strategy
 - Dynamic programming for small search space
 - Randomized for large search space
- Search space
 - Models alternative execution plans as operator trees
 - Different tree shapes
- Cost model (abstraction of execution system)
 - Physical schema info. (partitioning, indexes, etc.)
 - Statistics and cost functions

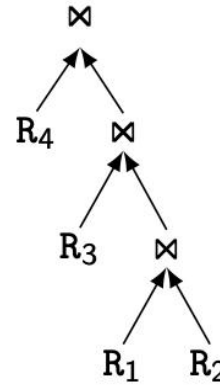


Operator Trees

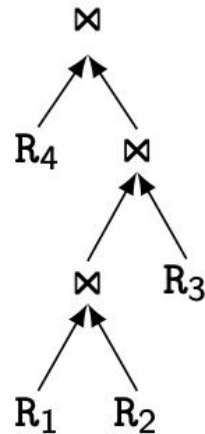
Left-deep



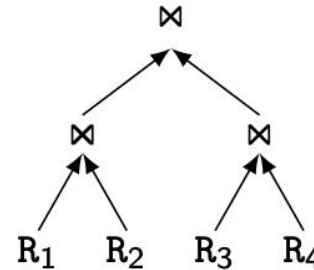
Right-deep



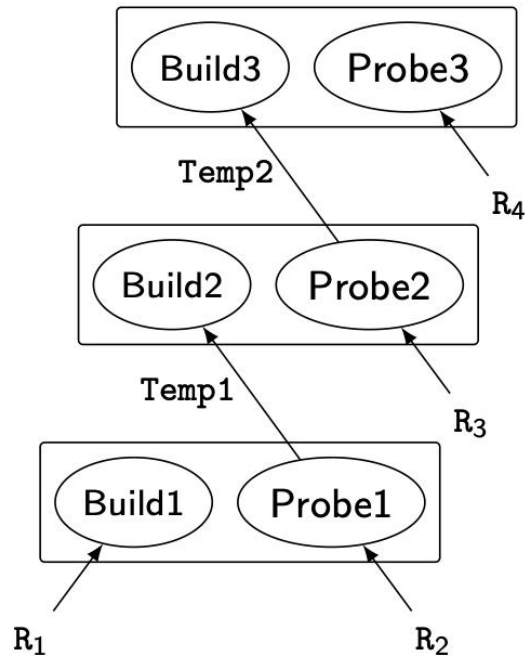
Zig-zag



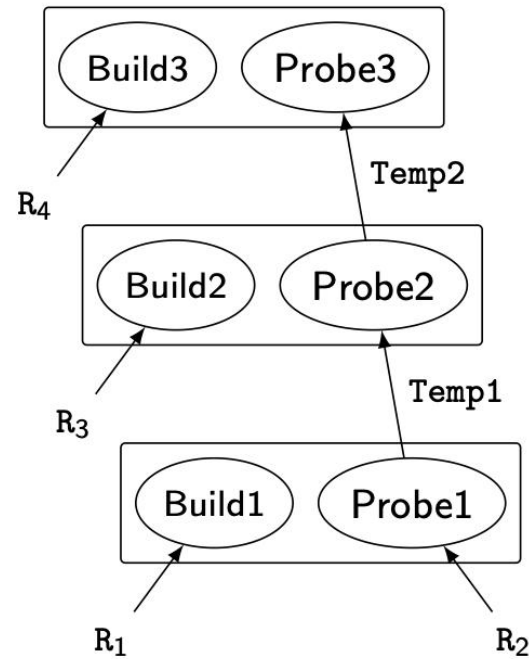
Bushy



Equivalent Hash-Join Trees with Different Scheduling



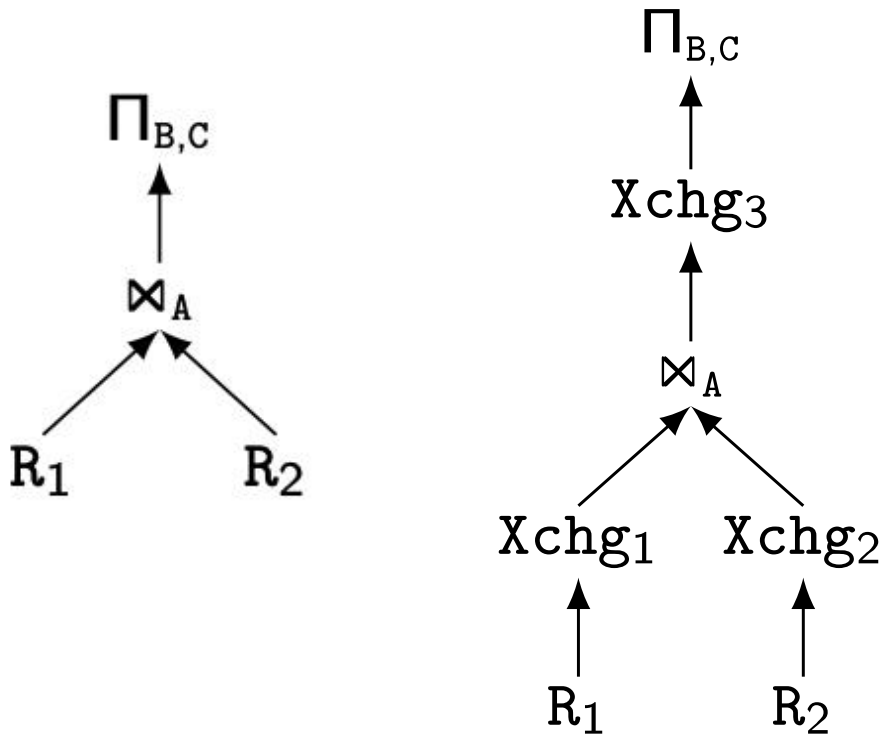
No pipeline



Pipeline of R_2 , Temp₁ and Temp₂



Operator Tree with Exchange Operator



$Xchg_1$: partition by $h(A)$

$Xchg_2$: partition by $h(A)$

$Xchg_3$: partition by $h(B, C)$

Cost Model

- Total time
 - Similar to distributed query optimization
 - Adds all CPU, I/O and **com.** Costs
- Response time
 - More involved as it must take into account pipelining
 - Schedule plan p in phases ph

$$respTime(p) = \sum_{ph \in p} (\max_{Op \in ph} (respTime(Op) + pipeDelay(Op)) + storeDelay(ph))$$

where

- ▶ $pipeDelay(Op)$ is the time necessary for the operator Op to deliver the first result tuples
- ▶ $storeDelay(ph)$ is the time necessary to store the result of phase ph



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

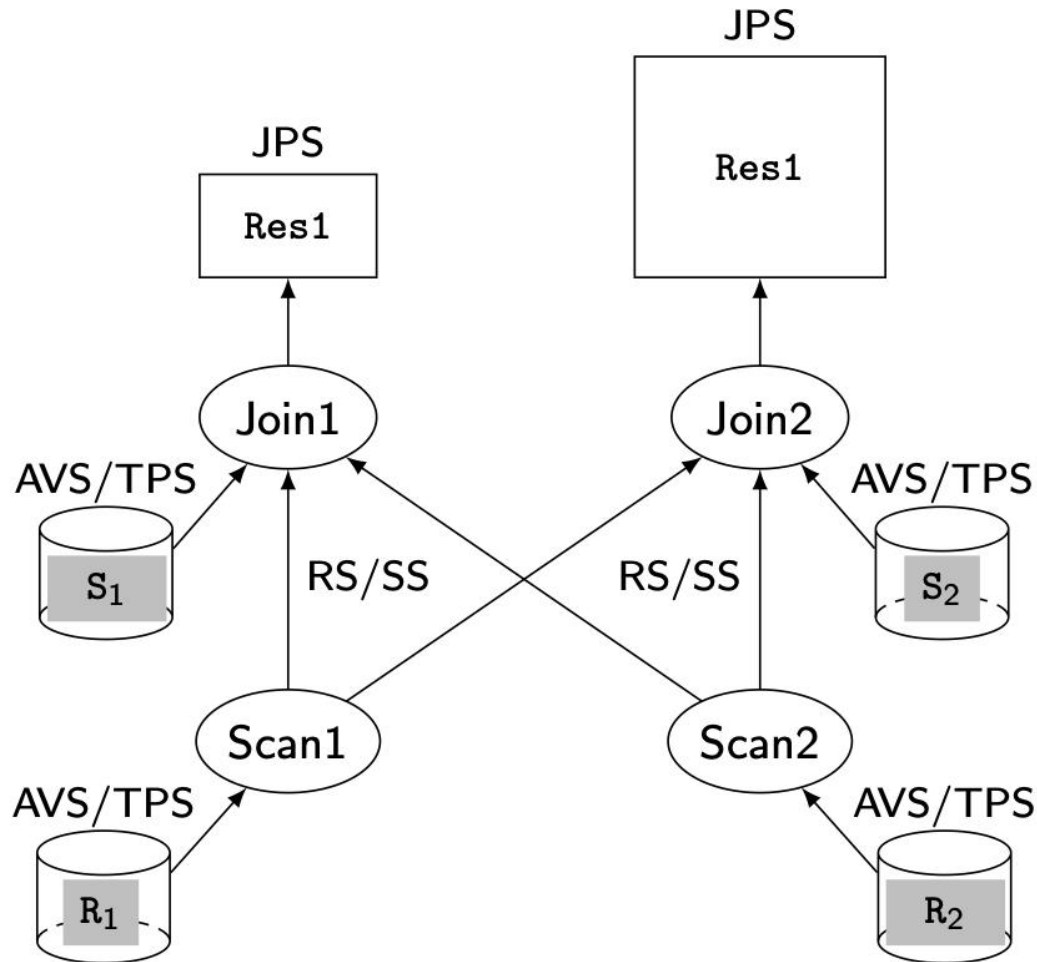


Load Balancing

- Initialization
- Interference
- Problems arise for intra-operator parallelism with *skewed* data distributions
 - Attribute data skew (AVS)
 - Tuple placement skew (TPS)
 - Selectivity skew (SS)
 - Redistribution skew (RS)
 - Join product skew (JPS)
- Solutions
 - Sophisticated parallel algorithms that deal with skew
 - Dynamic processor allocation (at execution time)

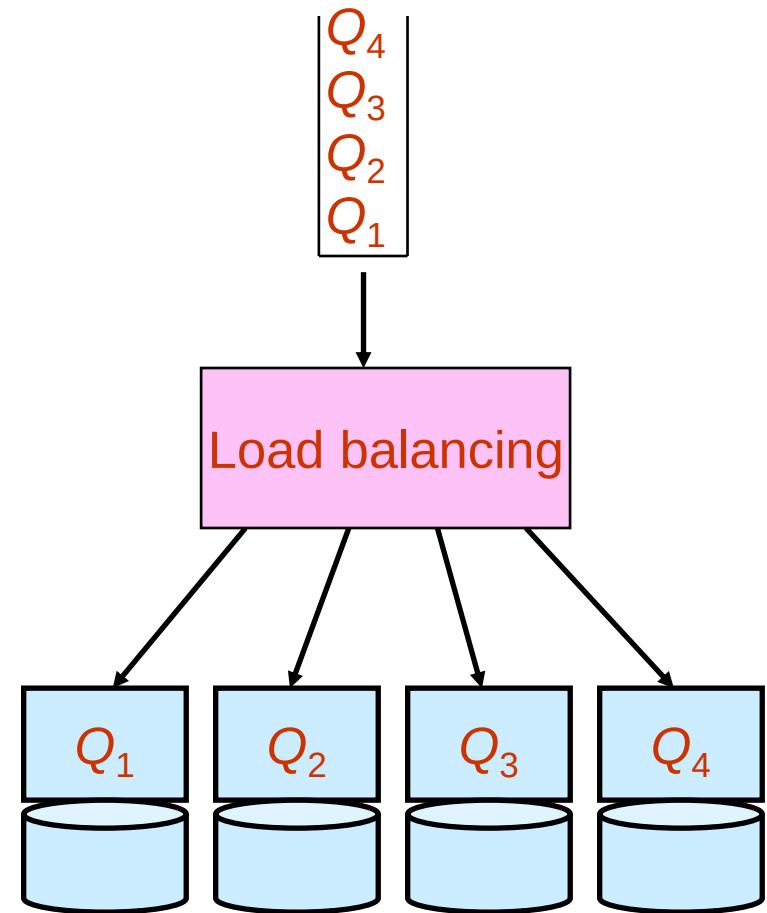


Data Skew Example



Load Balancing

- ❑ Choose the node to execute Q
 - ❑ Round robin
 - ❑ The least loaded
 - ▶ Need to get load information
- ❑ Failover
 - ❑ In case a node N fails, N 's queries are taken over by another node
 - ▶ Requires a copy of N 's data
- ❑ In case of interference
 - ❑ Data of an overloaded node are replicated to another node



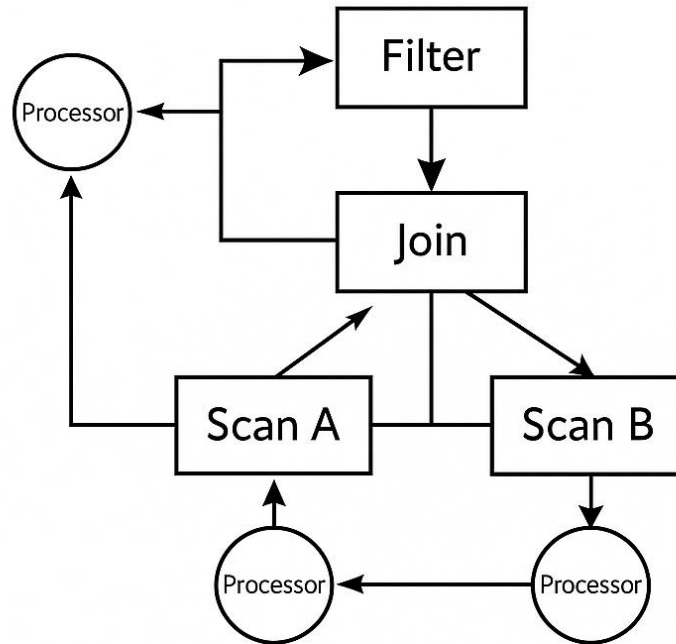
Dynamic Processing (DP) Execution Model

- A query is decomposed into self-contained units of sequential processing, each of which can be carried out by any processor
- Intuitively, a processor can migrate horizontally (intraoperator parallelism) and vertically (interoperator parallelism) along the query operators
- The parallel execution plan as produced by the optimizer includes operator scheduling constraints that express a partial order among the operators of the query



Dynamic Processing (DP) Execution Model

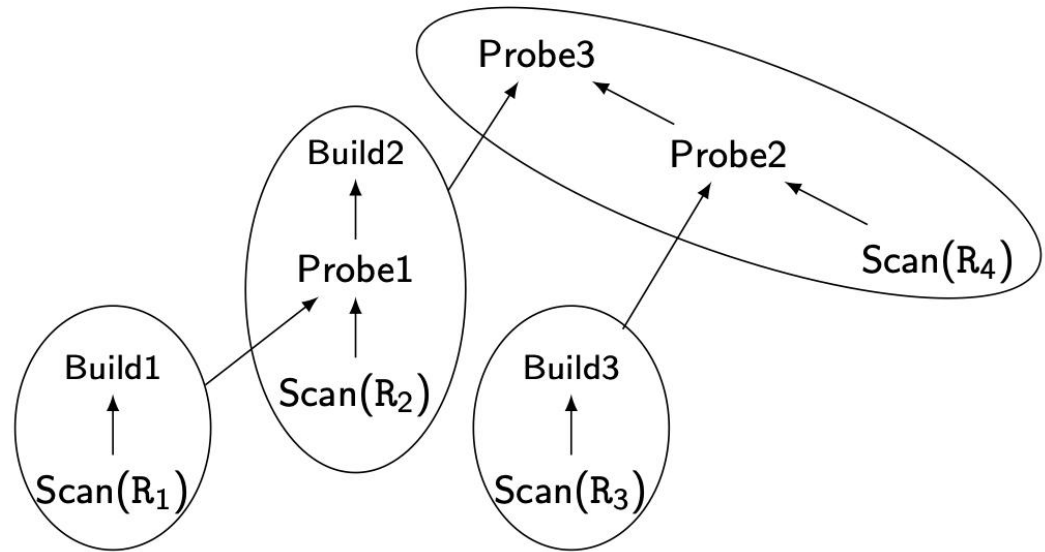
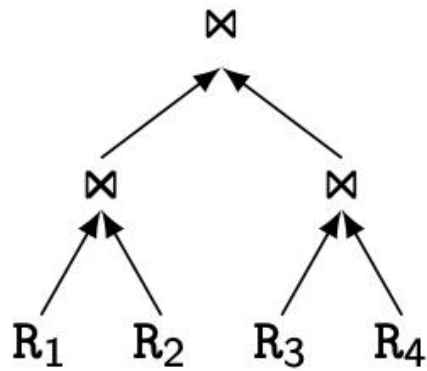
A query is decomposed into self-contained units of sequential processing, each of which can be carried out by any *processor*



Intuitively, a processor can migrate horizontally (*intraoperator parallelism*) and vertically (*interoperator parallelism*) along the query operators

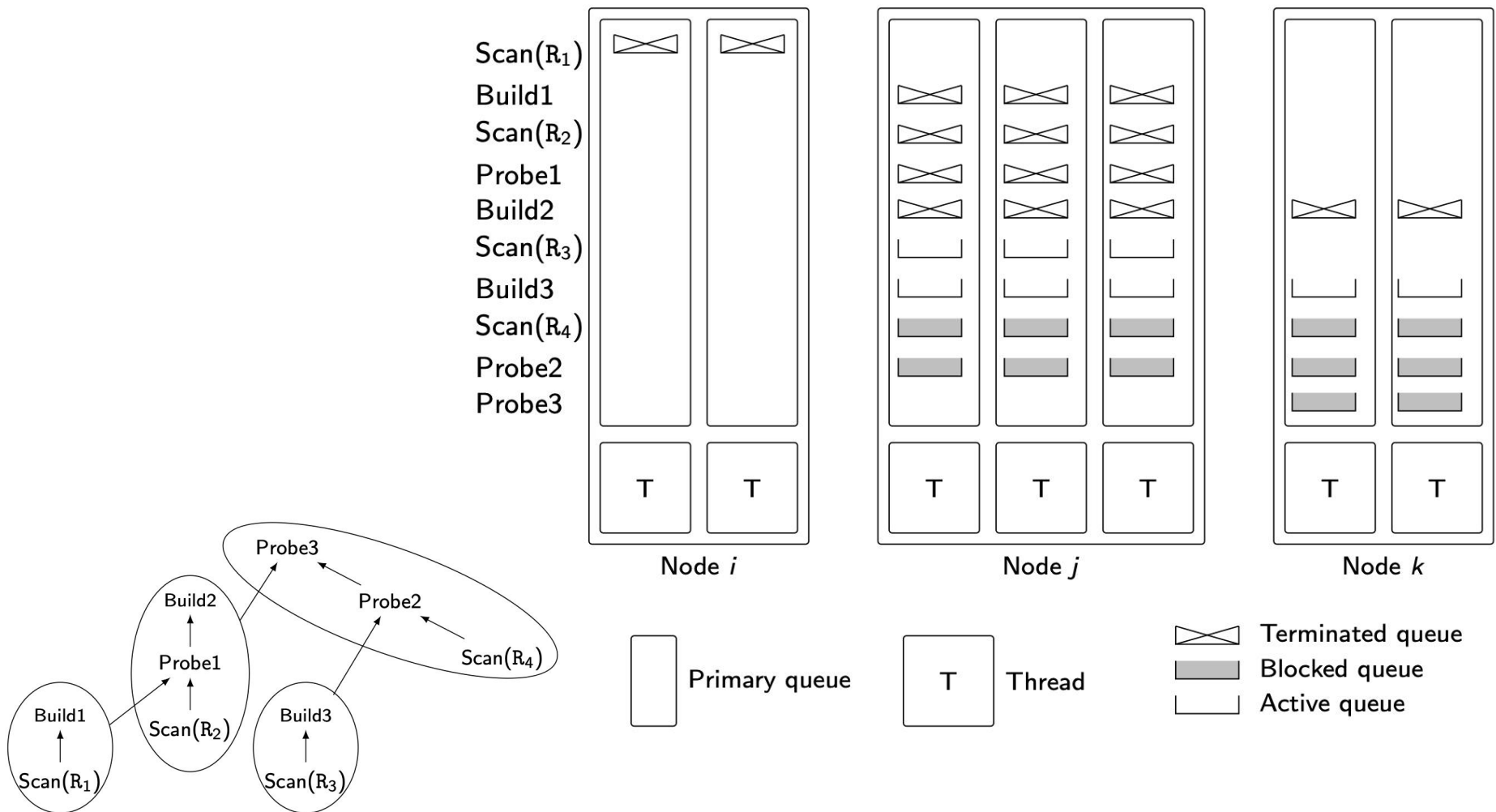
The parallel execution plan as produced by the optimizer

DP Example



Join tree with 4 pipelined chains

DP Example – snapshot of execution



Replication and Failover

□ Replication

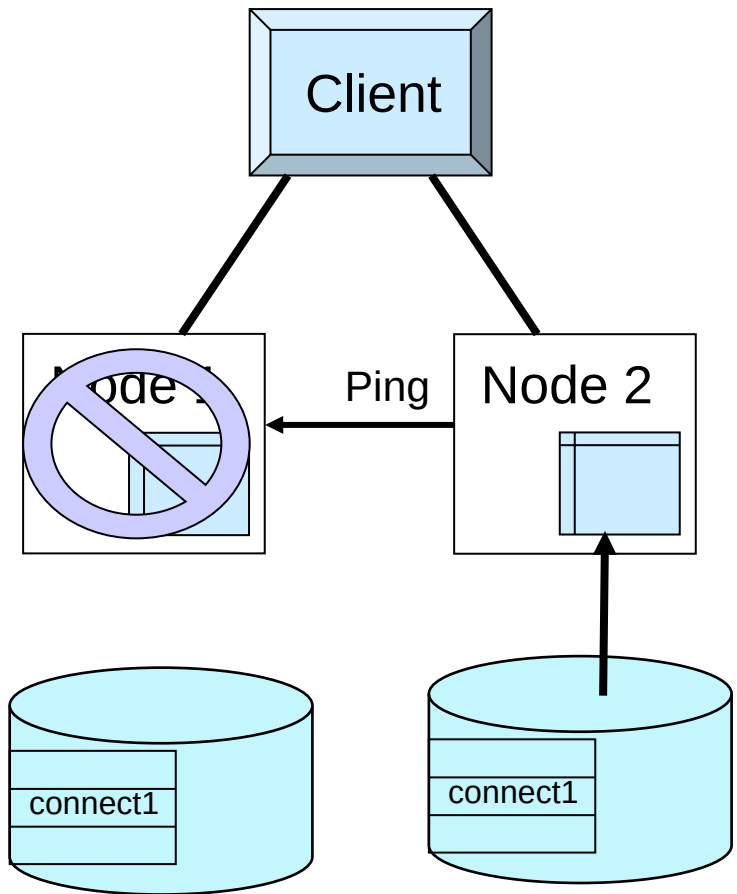
- The basis for fault-tolerance and availability

□ Failover

- On a node failure, another node detects and recovers the node's tasks

=>

Savepoints for long queries



Outline

- Parallel Database Systems
 - Parallel Architectures
 - Data Placement
 - Query Processing
 - Load Balancing and Fault-tolerance
 - Database Clusters

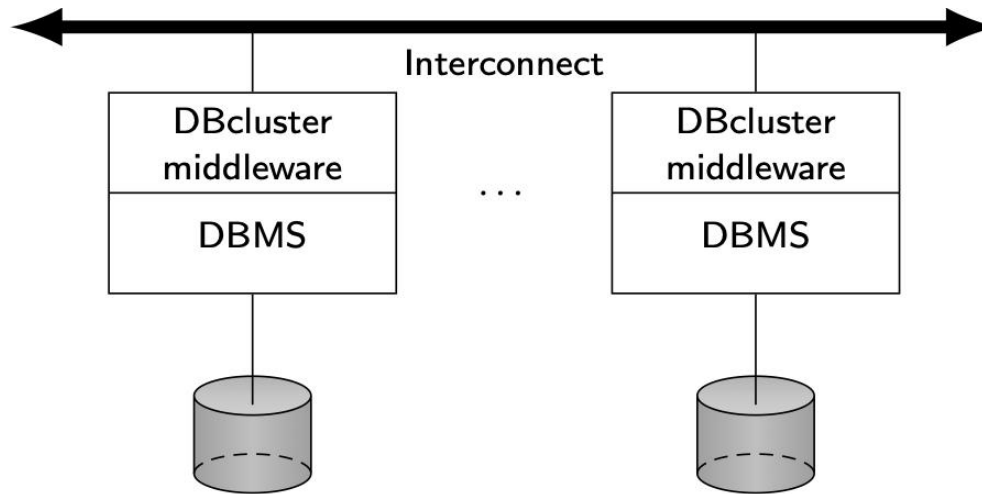


Motivations

- ❑ Database cluster = cluster of autonomous databases, each managed by an off-the-shelf DBMS
- ❑ Major difference with a parallel DBMS is the use of a “black-box” DBMS at each node
 - ❑ In general, same DBMS
- ❑ Since the DBMS source code is not necessarily available and cannot be changed to be “cluster-aware,” parallel data management capabilities must be implemented via middleware
- ❑ Examples: MySQL or PostgreSQL clusters



Database Cluster Architecture



- Middleware has several software layers
 - Transaction load balancer, replication manager, query processor, and fault-tolerance manager

Replication

- ❑ The fast interconnect and communication system can be exploited to support one-copy serializability
- ❑ Example with Preventive Replication
 - ❑ Uses FIFO reliable multicast (simple and efficient)
 - ❑ Principle
 - ▶ Each incoming transaction T has a chronological timestamp $ts(T) = C$, and is multicast to all other nodes where there is a copy.
 - ▶ At each node, a time delay is introduced before starting the execution of T . This delay corresponds to the upper bound of the time needed to multicast a message
 - ▶ When the delay expires, all transactions that may have committed before C are guaranteed to be received and executed before T , following the timestamp order (i.e., total order)

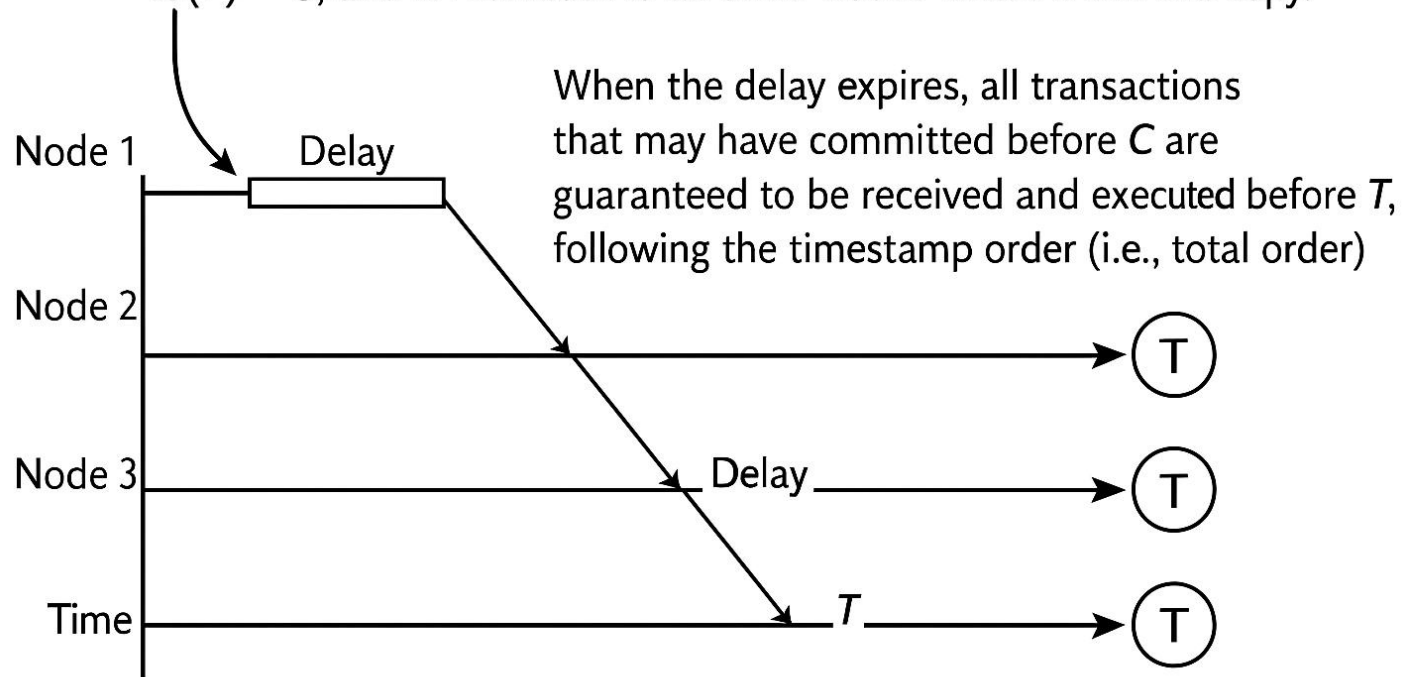


Replication

Example with Preventive Replication

Uses FIFO reliable multicast (simple and efficient)

Each incoming transaction T has a chronological timestamp $ts(T) = C$, and is multicast to all other nodes where there is a copy.



Load Balancing

- Query load balancing is easy
 - Since all copies are mutually consistent, any node that stores a copy of the data, e.g., the least loaded node, can be chosen at runtime by a conventional load balancing strategy
- Transaction load balancing
 - The total cost of transaction execution at all nodes may be high
 - By relaxing consistency, lazy replication can better reduce transaction execution cost and thus increase performance of both queries and transactions



Query Processing

- Interquery parallelism is naturally obtained as a result of load balancing and replication
 - Useful to increase the throughput of transaction-oriented applications and, to some extent, to reduce the response time of transactions and queries
- Intraquery parallelism is essential to further reduce response times of OLAP queries
 - More difficult than data partitioning because of black-box DBMSs
 - 2 solutions
 - ▶ Physical partitioning
 - ▶ Virtual partitioning



Physical Partitioning

- Similar to data partitioning in distributed databases except that the objective is to increase intraquery parallelism, not locality of reference
- Thus, depending on the query and relation sizes, the degree of partitioning should be much finer
- Under uniform data distribution, yields good intraquery parallelism and outperforms interquery parallelism
- However, it is static and thus very sensitive to data skew conditions and the variation of query patterns that may require periodic repartitioning



Virtual Partitioning

- Uses full replication (each relation is replicated at each node).
- Simple virtual partitioning (SVP)
 - Virtual partitions are dynamically produced for each query and intraquery parallelism is obtained by sending subqueries to different virtual partitions
 - To produce the different subqueries, the query processor adds predicates to the incoming query in order to restrict access to a subset of a relation, i.e., a virtual partition
 - Then, each DBMS that receives a subquery is forced to process a different subset of data items
 - Finally, the partitioned result needs to be combined by an aggregate query



SVP Example

- Consider

```
SELECT  PNO,  AVG (DUR)
FROM    WORKS
WHERE    SUM (DUR) > 200
GROUP BY PNO
```

- A generic subquery on a virtual partition is obtained by adding to Q's where clause the predicate

```
AND PNO >= 'P1' AND PNO < 'P2'
```



SVP Assessment

- ❑ Great flexibility for node allocation during query processing since any node can be chosen for executing a subquery
- ❑ Not all kinds of queries can benefit from SVP
 - ❑ Only queries without subqueries that access a fact table are ok
 - ❑ Queries with a subquery must be converted to have no subquery
 - ❑ Any other queries cannot benefit
- ❑ Limitations
 - ❑ Determining the best virtual partitioning attributes and value ranges can be difficult
 - ❑ Dependent on the underlying DBMS query capabilities
- ❑ Solution: Adaptive Virtual Partitioning

